

Chương 1

Từ Monolithic đến Microservice và DDD

1.1 Kiến trúc đơn khối (Monolithic Architecture)

1.1.1 Khái niệm và Bản chất Kỹ thuật

Kiến trúc Monolithic không chỉ đơn thuần là việc đóng gói mã nguồn vào một tệp tin thực thi (như .war hay .jar), mà còn đại diện cho một tư duy thiết kế tập trung. Trong mô hình này, toàn bộ logic nghiệp vụ, giao diện người dùng, và các thành phần truy xuất dữ liệu được liên kết chặt chẽ (tightly coupled) trong một đơn vị duy nhất.

Sự gắn kết này có nghĩa là các thành phần chia sẻ chung bộ nhớ, tài nguyên CPU và thường là một cơ sở dữ liệu quan hệ duy nhất. Bất kỳ thay đổi nhỏ nào trong một module cũng đòi hỏi việc biên dịch lại và triển khai lại toàn bộ hệ thống.

1.1.2 Cấu trúc Phân lớp điển hình

Đa số các hệ thống Monolithic hiện đại được tổ chức theo mô hình ****Layered Architecture**** (Kiến trúc phân lớp):

- **Presentation Layer:** Xử lý giao diện người dùng và các yêu cầu HTTP.
- **Business Logic Layer:** Chứa các quy tắc nghiệp vụ cốt lõi của hệ thống.
- **Data Access Layer (DAL):** Chịu trách nhiệm giao tiếp với cơ sở dữ liệu.

Mặc dù được phân lớp về mặt logic, nhưng ở cấp độ vật lý, chúng vẫn chạy chung một tiến trình (process).

1.1.3 Ưu điểm và Hạn chế trong Vận hành

Ưu điểm:

- **Đơn giản trong quản lý:** Quá trình kiểm thử tích hợp (End-to-End testing) diễn ra thuận lợi do không có độ trễ mạng giữa các thành phần.
- **Hiệu suất tối ưu:** Các lời gọi hàm trực tiếp trong bộ nhớ luôn nhanh hơn việc gọi qua API hoặc Message Broker.
- **Nhất quán dữ liệu (ACID):** Việc sử dụng một DB duy nhất giúp đảm bảo tính toàn vẹn dữ liệu thông qua các Transaction truyền thống.

Hạn chế (The Big Ball of Mud):

- **Rào cản về quy mô (Scalability):** Không thể mở rộng riêng biệt một module bị nghẽn (ví dụ: module xử lý thanh toán), buộc phải nhân bản toàn bộ ứng dụng, gây lãng phí tài nguyên.
- **Sự trì trệ về công nghệ (Technology Lock-in):** Việc nâng cấp một thư viện hoặc ngôn ngữ mới là cực kỳ rủi ro vì nó ảnh hưởng đến tất cả các phần còn lại.

1.2 Kiến trúc Dịch vụ nhỏ (Microservices Architecture)

1.2.1 Triết lý thiết kế và Đặc điểm cốt lõi

Khác với Monolithic, Microservices tuân thủ triết lý "Làm một việc và làm thật tốt" (Do one thing and do it well). Mỗi dịch vụ là một thực thể độc lập, có vòng đời phát triển và triển khai riêng biệt.

Đặc điểm quan trọng nhất là **Bao đóng dữ liệu (Data Encapsulation)**: Mỗi microservice sở hữu cơ sở dữ liệu riêng (Database per Service), ngăn chặn việc truy cập dữ liệu trực tiếp từ các dịch vụ khác, từ đó giảm thiểu sự phụ thuộc.

1.2.2 Hệ sinh thái và Các thành phần hỗ trợ

Để một hệ thống Microservices vận hành ổn định, cần có các thành phần hạ tầng phức tạp:

- **API Gateway:** Điểm tiếp nhận duy nhất cho client, chịu trách nhiệm định tuyến, xác thực và giới hạn lưu lượng.
- **Service Discovery:** Cho phép các dịch vụ tìm thấy nhau trong môi trường mạng động (ví dụ: Netflix Eureka, Consul).
- **Circuit Breaker:** Cơ chế ngăn chặn lỗi dây chuyền khi một dịch vụ gặp sự cố (ví dụ: Hystrix hoặc Resilience4j).

1.2.3 Thách thức về Quản trị và Nhất quán dữ liệu

Mặc dù mang lại sự linh hoạt, Microservices đối mặt với bài toán **Nhất quán sau cùng (Eventual Consistency)**. Do dữ liệu bị phân tán, việc thực hiện các giao dịch phân tán (Distributed Transactions) trở nên cực kỳ khó khăn, thường phải áp dụng các mô hình như **Saga Pattern** để điều phối.

1.3 Phân tích so sánh: Khi nào nên chuyển đổi?

Việc lựa chọn kiến trúc không dựa trên xu hướng, mà dựa trên "Complexity-Value Curve" (Đường cong giá trị và độ phức tạp).

- Với các dự án khởi nghiệp cần MVP nhanh, Monolithic là sự lựa chọn tối ưu.
- Khi số lượng lập trình viên tăng lên (trên 20-30 người) và các yêu cầu nghiệp vụ trở nên quá chằng chéo, việc phân rã sang Microservices trở thành yếu tố sống còn để duy trì tốc độ phát triển.

1.4 Thiết kế hướng tên miền (Domain-Driven Design - DDD)

1.4.1 Tại sao DDD là chìa khóa cho Microservices?

Một trong những lỗi lớn nhất khi triển khai Microservices là phân rã dịch vụ theo kỹ thuật (ví dụ: Service DB, Service UI). DDD giải quyết vấn đề này bằng cách tập trung vào nghiệp vụ cốt lõi.

1.4.2 Chiến lược thiết kế (Strategic Design)

Khái niệm quan trọng nhất là **Bounded Context** (Ngữ cảnh bao đóng). Nó xác định ranh giới logic mà trong đó một mô hình dữ liệu có ý nghĩa nhất định. Ví dụ: Từ "Sản phẩm" trong ngữ cảnh "Bán hàng" sẽ khác với "Sản phẩm" trong ngữ cảnh "Kho bãi". Việc xác định đúng các Bounded Context chính là bước đầu tiên để xác định ranh giới của các Microservices.

1.4.3 Context Mapping: Bản đồ tương tác

Context Map giúp mô tả mối quan hệ giữa các ngữ cảnh:

- **Shared Kernel:** Hai ngữ cảnh chia sẻ chung một phần mô hình.
- **Anticorruption Layer (ACL):** Lớp đệm giúp dịch vụ mới không bị ảnh hưởng bởi thiết kế lỗi thời của hệ thống cũ.

1.4.4 Thiết kế kỹ thuật (Tactical Design)

Để thực thi logic bên trong một Microservice, DDD cung cấp các công cụ:

- **Aggregate:** Một cụm các đối tượng dữ liệu được coi là một đơn vị để thay đổi dữ liệu.
- **Value Objects:** Các đối tượng không có định danh, chỉ chứa giá trị (như Địa chỉ, Tiền tệ).
- **Repositories:** Lớp trung gian giữa Domain và Data Mapping.

Chương 2

Phát triển Hệ thống E-Commerce dựa trên Microservices

2.1 Xác định yêu cầu hệ thống

Trong bối cảnh thị trường thương mại điện tử cạnh tranh khốc liệt, việc xây dựng một hệ thống có khả năng thích ứng nhanh và quy mô lớn là điều bắt buộc. Hệ thống được thiết kế để phục vụ cả khách hàng (Customer) và nhân viên quản trị (Staff), tích hợp trí tuệ nhân tạo (AI) để tối ưu hóa trải nghiệm người dùng.

2.1.1 Yêu cầu chức năng (Functional Requirements)

Hệ thống bao gồm các nhóm chức năng chính sau:

- **Quản lý người dùng và xác thực:** Cho phép khách hàng đăng ký, đăng nhập và quản lý thông tin cá nhân. Nhân viên có các quyền hạn riêng biệt theo vai trò (Role-based Access Control).
- **Quản lý danh mục sản phẩm:** Hiển thị và quản lý thông tin chi tiết các dòng Laptop và Mobile, bao gồm cấu hình kỹ thuật, giá cả và chương trình khuyến mãi.
- **Hệ thống giỏ hàng:** Cho phép người dùng thêm, bớt sản phẩm từ các dịch vụ khác nhau (Laptop/Mobile) vào một giỏ hàng nhất nhất quán.
- **Tư vấn thông minh (AI Chatbot):** Cung cấp khả năng trả lời tự động các thắc mắc về sản phẩm dựa trên cơ sở tri thức (Knowledge Base) được trích xuất từ cơ sở dữ liệu đồ thị.

2.1.2 Yêu cầu phi chức năng (Non-functional Requirements)

Để đảm bảo chất lượng dịch vụ cấp doanh nghiệp, hệ thống cần đáp ứng:

- **Tính mở rộng (Scalability):** Khả năng mở rộng độc lập các dịch vụ có tải cao như Laptop Service hoặc AI Service mà không ảnh hưởng đến toàn bộ hệ thống.
- **Tính sẵn sàng cao (High Availability):** Hệ thống phải hoạt động liên tục, giảm thiểu điểm chết duy nhất (Single Point of Failure) thông qua cơ chế replication và load balancing.
- **Tính bảo mật (Security):** Dữ liệu người dùng phải được mã hóa, các API được bảo vệ qua Gateway và cơ chế xác thực tập trung.
- **Khả năng bảo trì (Maintainability):** Mã nguồn được tổ chức theo cấu trúc chuẩn của Django và Microservices, giúp dễ dàng nâng cấp và sửa lỗi từng phần.

2.2 Phân rã hệ thống theo thiết kế hướng tên miền (DDD)

Việc phân rã hệ thống dựa trên nghiệp vụ thay vì kỹ thuật giúp đảm bảo tính bao đóng và giảm thiểu sự phụ thuộc chéo.

2.2.1 Xác định các Ngữ cảnh bao đóng (Bounded Contexts)

Dựa trên phân tích nghiệp vụ, hệ thống được chia thành 5 Bounded Contexts chính:

- **Customer Context:** Tập trung vào thực thể khách hàng và quy trình tương tác của họ với hệ thống (Login, Register, Cart).
- **Laptop Context:** Quản lý chuyên sâu về các sản phẩm máy tính xách tay với các đặc tính kỹ thuật riêng biệt (CPU, GPU, Screen).
- **Mobile Context:** Quản lý các thiết bị di động với các thuộc tính như Camera, dung lượng Pin.
- **Staff Context:** Quản lý nhân sự và các hoạt động quản trị nội bộ.
- **AI Context:** Cung cấp các khả năng thông minh, xử lý ngôn ngữ tự nhiên để tư vấn bán hàng.

2.2.2 Các nguyên tắc và ràng buộc kiến trúc

Hệ thống tuân thủ các ràng buộc nghiêm ngặt:

- **Database per Service:** Mỗi ngữ cảnh sở hữu một cơ sở dữ liệu riêng, đảm bảo tính cô lập hoàn toàn về dữ liệu.
- **Polyglot Persistence:** Sử dụng đa dạng các loại cơ sở dữ liệu (PostgreSQL cho dữ liệu quan hệ và Neo4j cho dữ liệu đồ thị phục vụ AI).
- **Giao tiếp qua API/Gateway:** Các dịch vụ không gọi trực tiếp cơ sở dữ liệu của nhau mà phải thông qua các giao diện lập trình ứng dụng (API).

2.3 Thiết kế và Triển khai Customer Service

Customer Service đóng vai trò là điểm chạm đầu tiên của người dùng, xử lý các nghiệp vụ liên quan đến định danh và quản lý trạng thái mua hàng.

2.3.1 Logic nghiệp vụ

- **Quản lý danh tính:** Xử lý quy trình đăng ký tài khoản mới và xác thực người dùng dựa trên thông tin định danh duy nhất.
- **Quản lý giỏ hàng tập trung:** Khởi tạo giỏ hàng riêng biệt cho mỗi khách hàng ngay khi đăng ký.
- **Xử lý vật phẩm đa dạng:** Cho phép thêm các loại sản phẩm khác nhau (Laptop, Mobile) vào chung một giỏ hàng bằng cách lưu trữ mã định danh và loại sản phẩm.

- **Cập nhật số lượng thông minh:** Tự động kiểm tra sự tồn tại của vật phẩm trong giỏ; nếu đã tồn tại, hệ thống sẽ cộng dồn số lượng thay vì tạo bản ghi mới.

2.3.2 Mô hình dữ liệu (Data Models)

Hệ thống sử dụng các thực thể chính sau:

- **Customer:**
 - name: Họ và tên khách hàng (String).
 - phone: Số điện thoại liên lạc (String).
 - email: Địa chỉ thư điện tử (Unique Email).
 - username: Tên đăng nhập (Unique String).
 - password: Mật khẩu đã được băm (Hash String).
- **Cart:**
 - customer: Liên kết 1-1 với thực thể Customer.
- **CartItem:**
 - item_id: ID của sản phẩm từ dịch vụ ngoại lai (Integer).
 - product_type: Loại sản phẩm (MOBILE hoặc LAPTOP).
 - quantity: Số lượng sản phẩm trong giỏ (Positive Integer).
 - cart: Liên kết N-1 với thực thể Cart.

2.3.3 Giao diện lập trình ứng dụng (API Endpoints)

Endpoint	Method	Tác dụng	Dữ liệu yêu cầu	Dữ liệu trả về
/register/	POST	Đăng ký tài khoản	Thông tin Customer	JSON khách hàng
/login/	POST	Xác thực người dùng	Username, Password	Thông tin khách hàng
/cart-items/	GET	Liệt kê vật phẩm	Token xác thực	List CartItems
/cart-items/	POST	Thêm vào giỏ	item_id, type, qty	CartItem chi tiết

2.4 Thiết kế và Triển khai Laptop Service

Dịch vụ quản lý kho dữ liệu và đặc tính kỹ thuật của các dòng máy tính xách tay.

2.4.1 Logic nghiệp vụ

- **Phân loại sản phẩm:** Tổ chức máy tính theo hãng sản xuất và danh mục sử dụng (Gaming, Văn phòng, v.v.).
- **Quản lý cấu hình chi tiết:** Lưu trữ các thông số phần cứng chuyên sâu phục vụ việc so sánh và tra cứu.
- **Đồng bộ hóa dữ liệu đồ thị:** Cung cấp dữ liệu để ánh xạ sang Neo4j, xây dựng mạng lưới liên kết giữa sản phẩm và các thành phần phần cứng.

2.4.2 Mô hình dữ liệu (Data Models)

- **Mô hình quan hệ (Relational Model):**
 - **Laptop:** name, img_url, price, discount, ram, cpu, gpu, screen.
 - **Manufacturer / Category:** Chỉ bao gồm trường name để định danh.
- **Mô hình đồ thị (Graph Model):**
 - **Nodes:** Laptop, CPU, GPU, RAM, Screen, Manufacturer.
 - **Relationships:** HAS_CPU, MADE_BY, BELONGS_TO.

2.4.3 Giao diện lập trình ứng dụng (API Endpoints)

Endpoint	Method	Tác dụng	Dữ liệu yêu cầu	Dữ liệu trả về
/laptops/	GET	Lấy danh sách sản phẩm	Tham số lọc (tùy chọn)	Danh sách Laptop
/laptops/{id}/	GET	Chi tiết sản phẩm	ID sản phẩm	Đối tượng Laptop
/manufacturers/	GET	Lấy danh sách hãng	Không	Danh sách hãng

2.5 Thiết kế và Triển khai Mobile Service

Quản lý dải sản phẩm thiết bị di động và các đặc thù công nghệ liên quan.

2.5.1 Logic nghiệp vụ

- **Quản lý thông số di động:** Tập trung vào các yếu tố người dùng quan tâm như Camera và hiệu năng xử lý.
- **Phân mảnh thị trường:** Hỗ trợ phân loại sản phẩm theo thương hiệu để người dùng dễ dàng tiếp cận.

2.5.2 Mô hình dữ liệu (Data Models)

- **Mobile:**

- name, price, discount, camera: Thuộc tính cơ bản và đặc thù.
- ram, cpu, gpu: Cấu hình hiệu năng.
- manufacturer, category: Tham chiếu thực thể cha.

2.5.3 Giao diện lập trình ứng dụng (API Endpoints)

Endpoint	Method	Tác dụng	Dữ liệu yêu cầu	Dữ liệu trả về
/mobiles/	GET	Danh sách điện thoại	Không	List Mobile
/mobiles/{id}/	GET	Xem chi tiết	ID	Chi tiết Mobile

2.6 Thiết kế và Triển khai Staff Service

Hệ thống quản trị dành cho nhân sự vận hành sàn thương mại điện tử.

2.6.1 Logic nghiệp vụ

- **Kiểm soát truy cập (RBAC):** Phân quyền nhân viên theo các vai trò khác nhau trong hệ thống.
- **Quản lý vận hành:** Cung cấp giao diện để nhân viên theo dõi trạng thái hệ thống và dữ liệu sản phẩm.

2.6.2 Mô hình dữ liệu (Data Models)

- **Staff:**

- staff_id: Mã nhân viên duy nhất.
- name, phone, email: Thông tin liên lạc cá nhân.
- username, password: Thông tin đăng nhập.
- role: Vai trò (Admin, Manager, v.v.).

2.6.3 Giao diện lập trình ứng dụng (API Endpoints)

Endpoint	Method	Tác dụng	Dữ liệu yêu cầu	Dữ liệu trả về
/login/	POST	Đăng nhập quản trị	Credentials	Staff Profile

2.7 Thiết kế và Triển khai AI Service

Thành phần thông minh hỗ trợ tư vấn bán hàng tự động dựa trên tri thức thực tế.

2.7.1 Logic nghiệp vụ

- **Truy vấn tri thức thời gian thực:** Trích xuất dữ liệu từ Neo4j để đảm bảo chatbot luôn nắm bắt được kho hàng thực tế.
- **Xử lý ngôn ngữ tự nhiên (NLP):** Sử dụng LLM để hiểu ý định người dùng và chuyển đổi dữ liệu thô từ database thành câu trả lời tự nhiên.
- **Giới hạn phạm vi tư vấn (Guardrailing):** Đảm bảo AI chỉ tư vấn các sản phẩm có sẵn trong danh mục hệ thống.

2.7.2 Mô hình tri thức (Knowledge Base)

Dịch vụ này không sở hữu cơ sở dữ liệu quan hệ mà tương tác trực tiếp với đồ thị tri thức:

- **Nodes:** Laptop, Mobile, CPU, GPU, RAM, Manufacturer.
- **Relationships:** Ánh xạ các đặc tính kỹ thuật vào thực thể sản phẩm để AI có thể "suy luận" và so sánh cấu hình.

2.7.3 Giao diện lập trình ứng dụng (API Endpoints)

Endpoint	Method	Tác dụng	Dữ liệu yêu cầu	Dữ liệu trả về
/chat/	POST	Tư vấn sản phẩm	message, type	Câu trả lời từ AI

2.8 Kiến trúc API Gateway và Điều phối

Để che giấu sự phức tạp của hệ thống microservices phía sau, API Gateway được triển khai như một facade duy nhất. Nó chịu trách nhiệm proxy các yêu cầu đến đúng dịch vụ mục tiêu dựa trên đường dẫn URL, đồng thời xử lý các vấn đề xuyên suốt (cross-cutting concerns) như timeout và headers mapping.